

What the difference between wide and narrow transformation

Understanding **narrow** and **wide transformations** is one of the most important Spark interview topics because it explains **how Spark executes your code**.

What is a Transformation?

A transformation is an operation that **creates a new RDD or DataFrame** from an existing one.

Examples:

```
df.filter(...)  
df.select(...)  
df.join(...)  
df.groupBy(...)
```

Transformations are **lazy**, meaning Spark does **not** execute them immediately. It builds an execution plan (DAG) and executes it only when an action (such as `show()` or `count()`) is called.

Narrow Transformation

A **narrow transformation** is one where:

Each output partition depends on only one input partition.

No data needs to move between executors.

Visualization

Input Partitions

```
P1 → P1  
P2 → P2  
P3 → P3  
P4 → P4
```

Each partition is processed independently.

Example

```
df.filter(df.age > 18)
```

Suppose the data is:

P1

Ali 20
Sara 15
P2

John 30
Omar 17

After filtering:

P1

Ali 20
P2

John 30

Each partition filters its own data. No communication with other partitions is required.

Common Narrow Transformations

map()

```
rdd.map(lambda x: x * 2)
```

filter()

```
df.filter(df.salary > 5000)
```

select()

```
df.select("name", "salary")
```

withColumn()

```
df.withColumn("tax", df.salary * 0.1)
```

drop()

```
df.drop("address")
```

union()

```
df1.union(df2)
```

(When no shuffle is required.)

coalesce()

```
df.coalesce(4)
```

Usually narrow because it merges partitions without a full shuffle.

Characteristics of Narrow Transformations

-  No shuffle
 -  Faster
 -  Less network traffic
 -  Less disk I/O
 -  Process each partition independently
 -  Stay within the same stage
-

Wide Transformation

A **wide transformation** is one where:

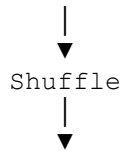
An output partition depends on multiple input partitions.

Spark must **shuffle** data across the cluster.

Visualization

Before

P1
P2
P3
P4



P1
P2
P3
P4

Rows move between partitions.

Example

```
df.groupBy("country").sum("sales")
```

Input:

P1

Egypt
USA
Egypt
P2

USA
Egypt
USA

Spark cannot compute totals independently because all "Egypt" rows must be together.

It shuffles the data:

P1

Egypt
Egypt
Egypt
P2

USA
USA
USA

Then performs the aggregation.

Common Wide Transformations

groupBy()

```
df.groupBy("country").count()
```

join()

```
customers.join(orders, "customer_id")
```

distinct()

```
df.distinct()
```

dropDuplicates()

```
df.dropDuplicates()
```

orderBy() / **sort()**

```
df.orderBy("salary")
```

repartition()

```
df.repartition(10)
```

Always performs a shuffle.

reduceByKey() (RDD)

```
rdd.reduceByKey(lambda x, y: x + y)
```

`aggregateByKey()`

`rdd.aggregateByKey(...)`

Characteristics of Wide Transformations

- ☒ Shuffle required
 - ☒ Network communication
 - ☒ More disk I/O
 - ☒ Slower than narrow transformations
 - ☒ Creates a new stage in the execution plan
-

Narrow vs. Wide

Narrow

Executor 1

P1 → Filter → P1

P2 → Filter → P2

No data movement.

Wide

Executor 1

Executor 2

P1 ————┐
P2 ————┤ → Shuffle → New Partitions
P3 ————┘

Rows move between executors.

Performance Comparison

Feature	Narrow	Wide
Shuffle	✗ No	✓ Yes
Network transfer	✗ No	✓ Yes
Speed	Faster	Slower
New stage	✗ Usually no	✓ Yes
Parallelism	High	Depends on shuffle
Resource usage	Lower	Higher

Interview Examples

Narrow

```
df.filter(df.age > 20)
  .select("name", "salary")
  .withColumn("tax", df.salary * 0.1)
```

Spark can execute these operations in the **same stage** because they don't require shuffling.

Wide

```
df.filter(df.age > 20) \
  .groupBy("country") \
  .sum("salary")
```

Execution flow:

```
Filter
  ↓
(No shuffle)
  ↓
GroupBy
  ↓
Shuffle
  ↓
Aggregation
```

The `groupBy()` introduces a **shuffle**, which ends one stage and starts another.

Common Interview Question

Q: Why are wide transformations slower than narrow transformations?

Answer:

Because wide transformations require a **shuffle**, where Spark redistributes data across partitions and executors. Shuffling involves network communication, disk I/O, and synchronization, making it one of the most expensive operations in Spark.

Summary Table

Narrow Transformations	Wide Transformations
<code>map()</code>	<code>groupBy()</code>
<code>filter()</code>	<code>join()</code>
<code>select()</code>	<code>orderBy()</code> / <code>sort()</code>
<code>withColumn()</code>	<code>distinct()</code>
<code>drop()</code>	<code>dropDuplicates()</code>
<code>flatMap()</code>	<code>repartition()</code>
<code>mapPartitions()</code>	<code>reduceByKey()</code> (RDD)
<code>coalesce()</code> (default)	<code>aggregateByKey()</code> (RDD)

Rule to Remember

- **Narrow transformation:** Each output partition depends on **one input partition**, so **no shuffle** is needed.
- **Wide transformation:** An output partition depends on **multiple input partitions**, so **a shuffle** is required.